

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



PROJECTE DE TECNOLOGIES DE LA INFORMACIÓ (PTI)

Informe Pràctica 3

Microserveis amb Kubernetes

0/

Autor Gabriel Calvo Catalán

Curs 2026-2027, quadrimestre de tardor

Data de lliurament 2/10/2026

1. Informació sobre l'ús d'IA

1) Com has aplicat la IA?

a), b), e), h). M'ha ajudat a entendre conceptes de Kubernetes (Deployment, Service, Ingress, HPA), a resoldre algun error i a polir l'informe.

2) Quines eines has fet servir?

b) Claude, per a dubtes de l'enunciat i per revisar la redacció.

3) Quin ordre has fet servir?

b) Primer ho he fet jo i després la IA ho ha completat/polit.

4) Mètode per al codi

a) Xat habitual. Els manifests YAML els he generat amb kubectl i editat jo mateix.

5) Has fet servir agents?

No.

6) Temps estalviat

Aproximadament un 20-25%.

7) Millora de la qualitat

Sí, sobretot a l'hora d'interpretar els resultats (per exemple, el comportament de l'HPA).

8) Impacte en l'aprenentatge

a) Positiu: aprenc de manera més eficient.

9) Ús recomanat

Fer servir la IA per entendre els conceptes i els errors de kubectl, però executar tots els passos un mateix per veure com respon el clúster.

2. Explicació tècnica

2.1 Entorn

La pràctica s'ha fet en un equip Ubuntu del laboratori amb Minikube com a clúster local de Kubernetes i kubectl per gestionar-lo. S'ha reutilitzat l'API carrental i el Dockerfile de la pràctica de Web APIs, construint la imatge amb nom i versió carrental:1.0.

2.2 Desplegament de carrental amb rèpliques (3.1)

S'ha creat el Deployment carrental amb 3 rèpliques a partir de la imatge carrental:1.0 i s'ha exposat com a Service de tipus NodePort al port 8080.

```
kubectl create deployment carrental --image=carrental:1.0 --replicas=3
kubectl expose deployment/carrental --type="NodePort" --port 8080
```

Com que cada Pod guarda les dades en un fitxer JSON propi, i el Service reparteix les peticions entre les 3 rèpliques, la llista de lloguers retornada pot variar d'una petició a una altra segons quin Pod respon. A més, les dades es perden si el Pod es reinicia, ja que els Pods són efímers. Per evitar-ho caldria un PersistentVolume o una base de dades compartida.

2.3 Minikube Dashboard (3.2)

S'ha obert el dashboard amb minikube dashboard i s'han revisat Deployments, Pods i Services.

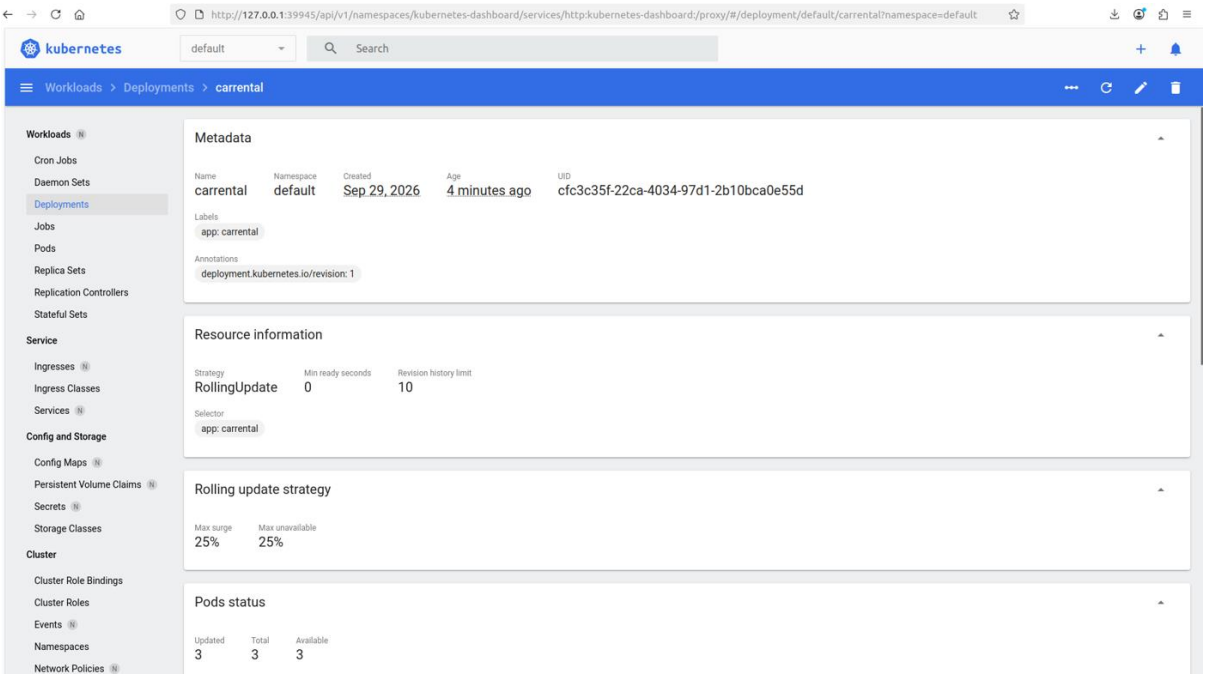


Figura 1. Detall del Deployment carrental: estratègia RollingUpdate i 3/3 Pods disponibles.

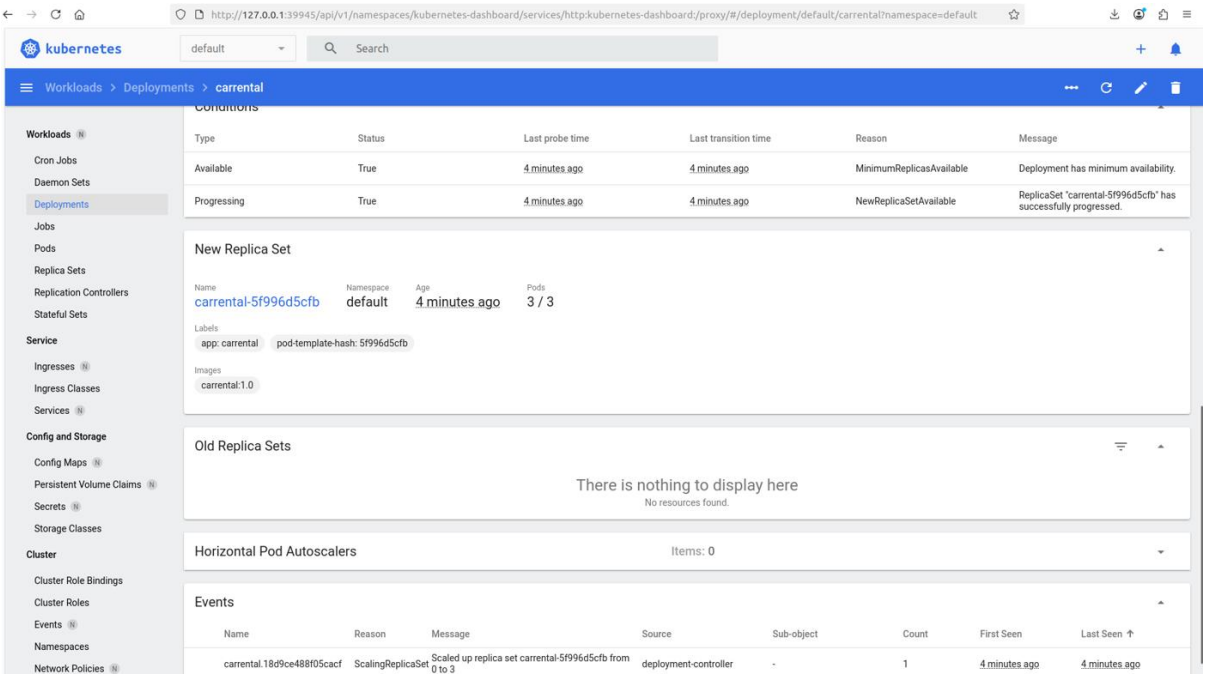


Figura 2. ReplicaSet de carrental amb 3/3 Pods i imatge carrental:1.0.

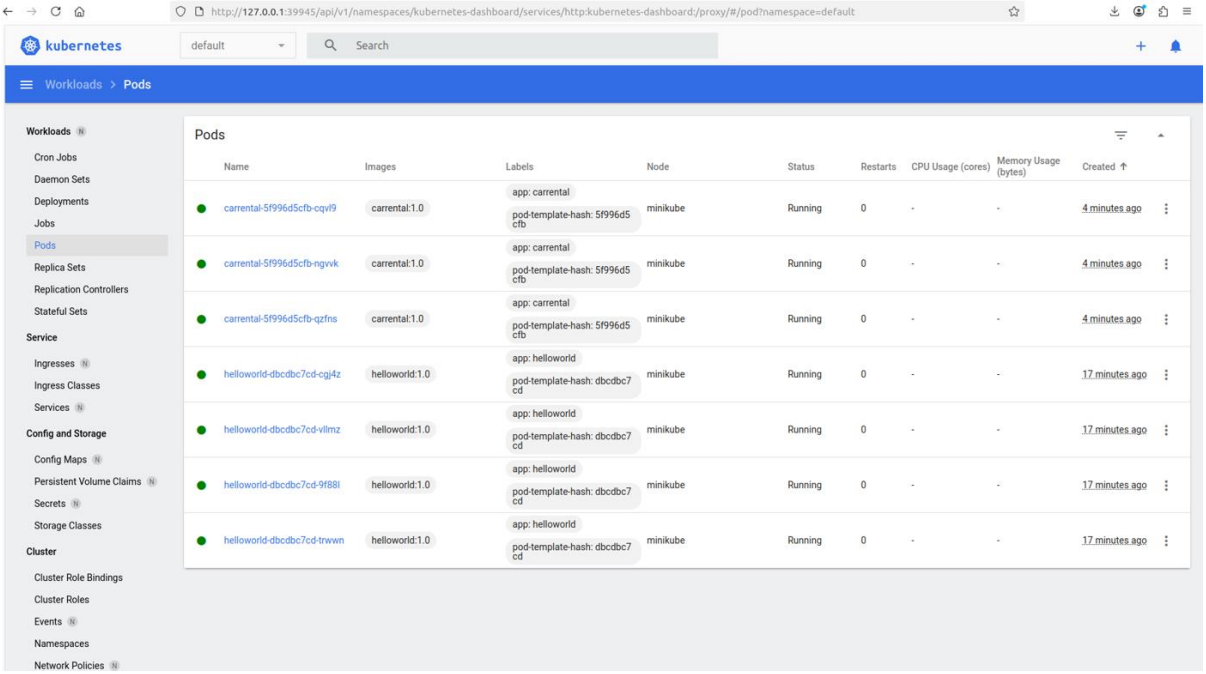


Figura 3. Llista de Pods: 3 rèpliques de carrental i 4 de helloworld, totes Running.

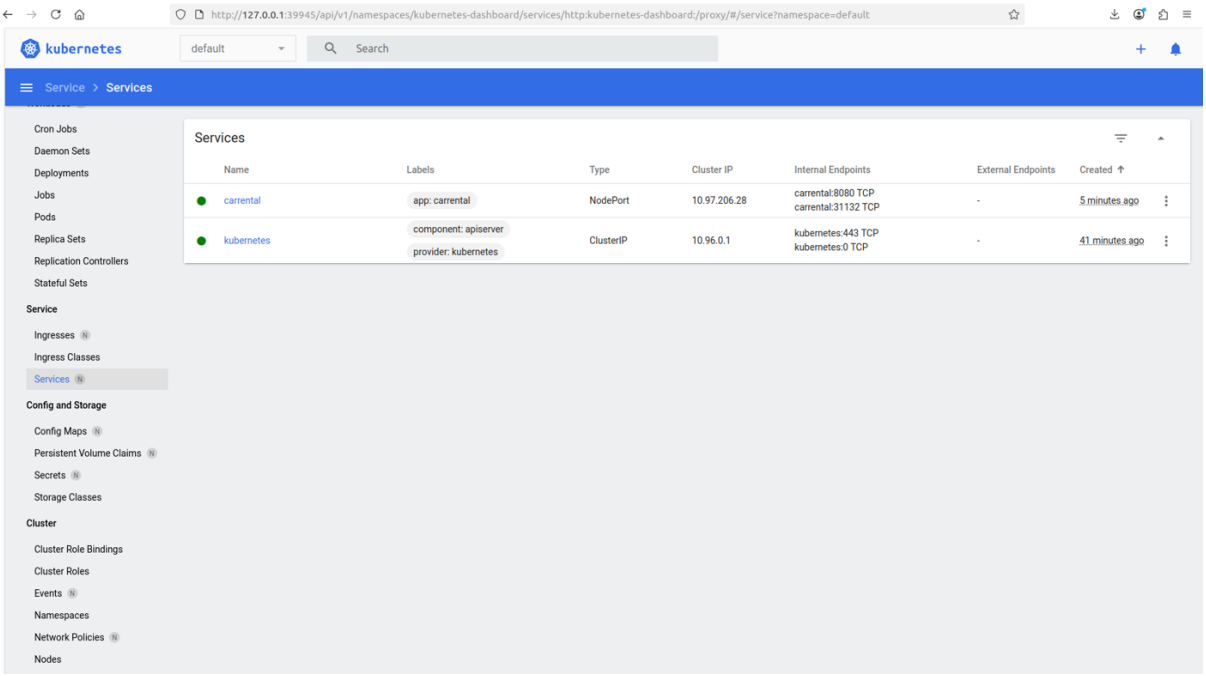


Figura 4. Services: carrental és de tipus NodePort (8080 intern, 31132 extern).

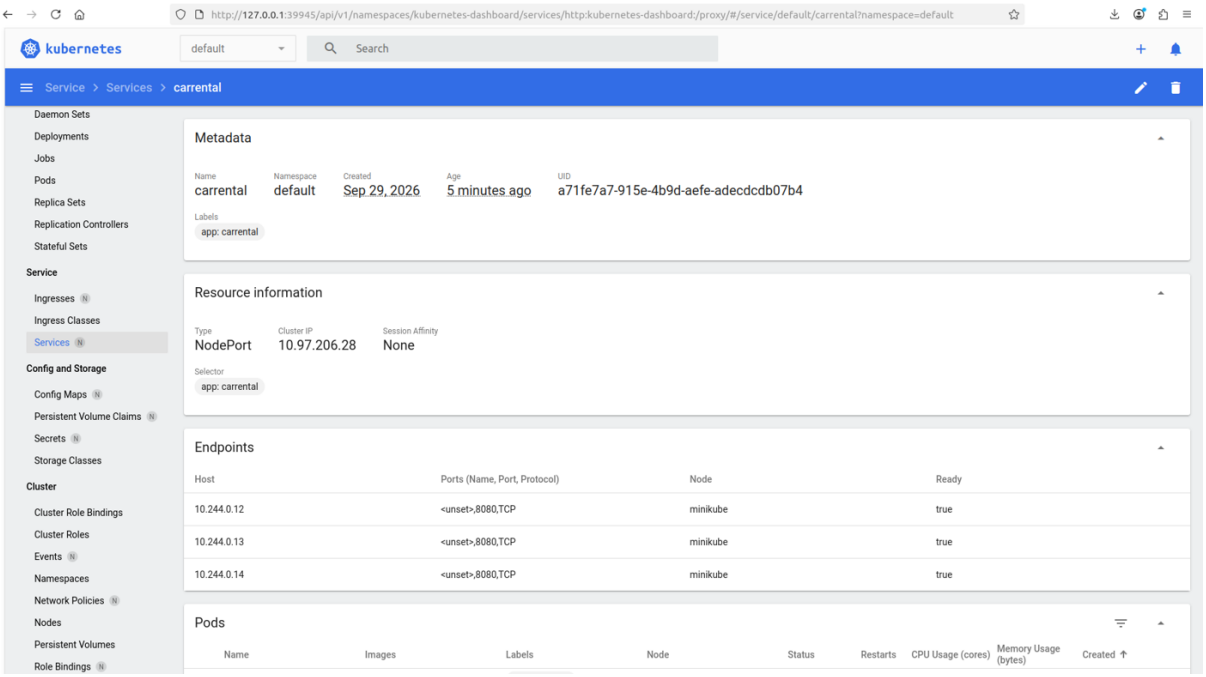


Figura 5. Detall del Service carrental amb els 3 endpoints (un per Pod).

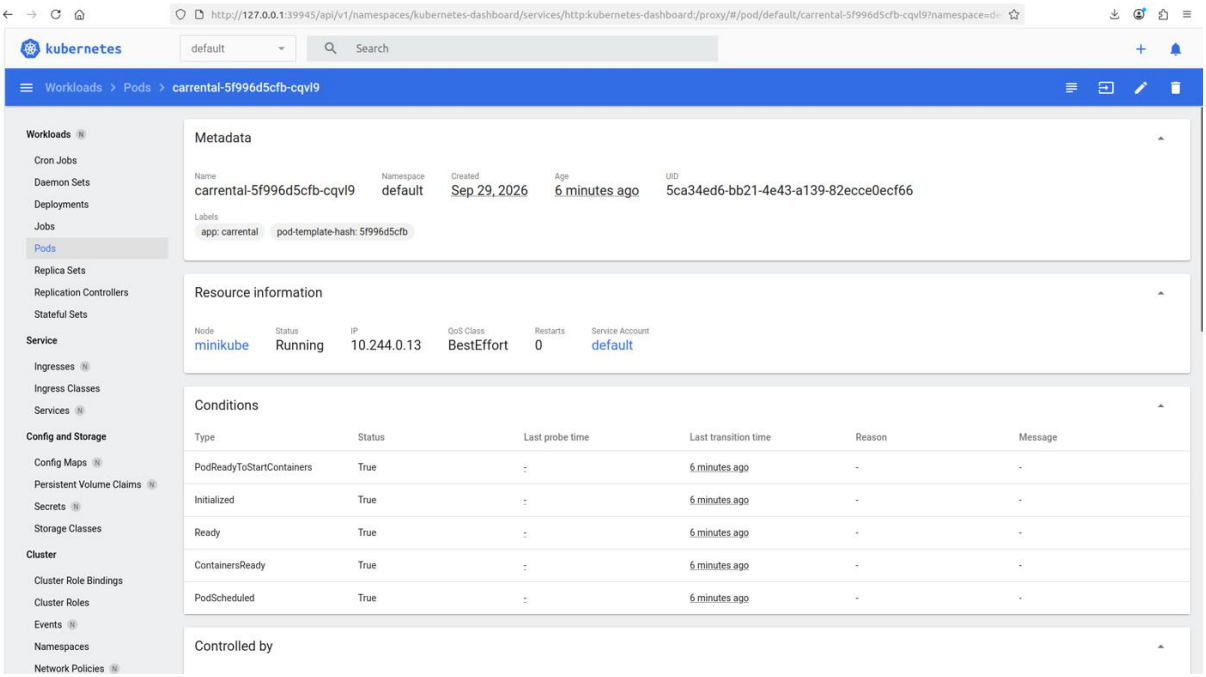


Figura 6. Detall d'un Pod de carrental: IP interna, estat Running i condicions.

Es veu clarament la jerarquia Deployment → ReplicaSet → Pods, i com el Service agrupa els Pods mitjançant el selector app: carrental.

2.4 Manifests YAML (3.3)

S'ha obtingut el manifest del Deployment, s'ha esborrat i s'ha tornat a crear amb kubectl apply. Després s'ha fet el mateix amb el Service.

```
kubectl get deployments/carrental -o=yaml > carrentaldeployment.yaml
kubectl delete deployment carrental
kubectl apply -f carrentaldeployment.yaml
```

```
alumne@a5s103pc25:~/pti-master-microservices/microservices$ cat carrentaldeployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: "2026-09-29T13:38:57Z"
  generation: 1
  labels:
    app: carrental
  name: carrental
  namespace: default
  resourceVersion: "2691"
  uid: cfc3c35f-22ca-4034-97d1-2b10bca0e55d
spec:
  progressDeadlineSeconds: 600
  replicas: 3
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: carrental
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: carrental
    spec:
      containers:
      - image: carrental:1.0
        imagePullPolicy: IfNotPresent
        name: carrental
        ports:
        - containerPort: 8080
          protocol: TCP
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
      dnsPolicy: ClusterFirst
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      terminationGracePeriodSeconds: 30
status:
  availableReplicas: 3
  conditions:
  - lastTransitionTime: "2026-09-29T13:38:58Z"
    lastUpdateTime: "2026-09-29T13:38:58Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  - lastTransitionTime: "2026-09-29T13:38:57Z"
    lastUpdateTime: "2026-09-29T13:38:58Z"
    message: ReplicaSet "carrental-5f996d5cfb" has successfully progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
  observedGeneration: 1
  readyReplicas: 3
  replicas: 3
  terminatingReplicas: 0
  updatedReplicas: 3
```

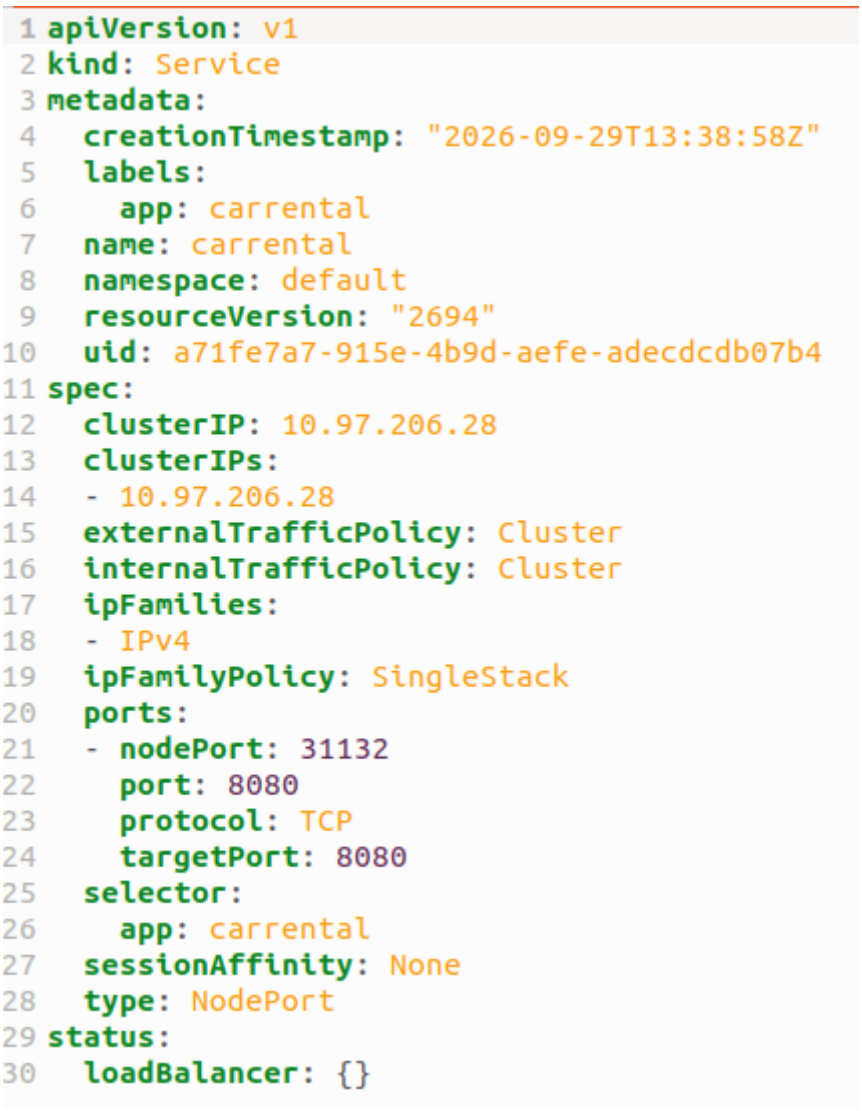
Figura 7. Manifest del Deployment carrental.

```
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl delete deployment carrental
deployment.apps "carrental" deleted from default namespace
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl apply -f carrentaldeployment.yaml
deployment.apps/carrental created
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl get pods
NAME                          READY   STATUS        RESTARTS   AGE
carrental-5f996d5cfb-cqvl9    1/1     Terminating   0          8m59s
carrental-5f996d5cfb-gfnnc    1/1     Running       0          7s
carrental-5f996d5cfb-l4nkg    1/1     Running       0          7s
carrental-5f996d5cfb-ngvvk    1/1     Terminating   0          8m59s
carrental-5f996d5cfb-qzfns    1/1     Terminating   0          8m59s
carrental-5f996d5cfb-vhbpb    1/1     Running       0          7s
helloworld-dbcdbc7cd-9f88l    1/1     Running       0          22m
helloworld-dbcdbc7cd-cgj4z    1/1     Running       0          21m
helloworld-dbcdbc7cd-trwwn    1/1     Running       0          22m
helloworld-dbcdbc7cd-vllmz    1/1     Running       0          21m
```

Figura 8. Esborrat i recreació del Deployment: els Pods antics passen a Terminating i se'n creen de nous.

```
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl get services/carrental -o=yaml > carrentalservice.yaml
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl delete service carrental
service "carrental" deleted from default namespace
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl apply -f carrentalservice.yaml
service/carrental created
alumne@a5s103pc25:~/pti-master-microservices/microservices$ minikube service carrental --url
http://192.168.49.2:31132
```

Figura 9. Recreació del Service des del YAML i obtenció de la URL.



```yaml
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: "2026-09-29T13:38:58Z"
  labels:
    app: carrental
  name: carrental
  namespace: default
  resourceVersion: "2694"
  uid: a71fe7a7-915e-4b9d-aefe-adecdcdb07b4
spec:
  clusterIP: 10.97.206.28
  clusterIPs:
  - 10.97.206.28
  externalTrafficPolicy: Cluster
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - nodePort: 31132
    port: 8080
    protocol: TCP
    targetPort: 8080
  selector:
    app: carrental
  sessionAffinity: None
  type: NodePort
status:
  loadBalancer: {}
```

Figura 10. Manifest del Service carrental.

Comparació: el manifest del Deployment descriu què s'executa (imatge, rèpliques, port del contenidor, estratègia d'actualització i plantilla dels Pods), mentre que el del Service descriu com s'hi accedeix (tipus NodePort, port 8080, nodePort 31132 i selector app: carrental). Tots dos inclouen camps generats pel clúster (uid, resourceVersion, creationTimestamp i status) que no caldria incloure en un manifest escrit a mà.

2.5 Ingress (3.4)

S'ha activat l'addon ingress i s'ha tornat a exposar carrental com a ClusterIP (només accessible dins el clúster). Després s'ha creat un Ingress que redirigeix el host carrental.local al Service.

```
minikube addons enable ingress
kubectl delete service/carrental
kubectl expose deployment/carrental --type="ClusterIP" --port 8080
kubectl apply -f carrental-ingress.yaml
```

```
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl delete service/carrental
service "carrental" deleted from default namespace
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl get services/carrental -o=yaml
^C
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl expose deployment/carrental --type="ClusterIP" --port 8080
service/carrental exposed
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl get services/carrental -o=yaml
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: "2026-09-29T14:00:08Z"
  labels:
    app: carrental
  name: carrental
  namespace: default
  resourceVersion: "4108"
  uid: 7fe63416-062c-4347-a462-acef4be0eaad
spec:
  clusterIP: 10.110.107.196
  clusterIPs:
  - 10.110.107.196
  internalTrafficPolicy: Cluster
  ipFamilies:
  - IPv4
  ipFamilyPolicy: SingleStack
  ports:
  - port: 8080
    protocol: TCP
    targetPort: 8080
  selector:
    app: carrental
  sessionAffinity: None
  type: ClusterIP
status:
  loadBalancer: {}
```

Figura 11. Service carrental recreat com a ClusterIP.

Open | carrental-ingress.yaml — ~/pti-master-microservices/microservices | Save

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: carrental
  namespace: default
  labels:
    app: carrental
spec:
  rules:
    - host: carrental.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: carrental
                port:
                  number: 8080
```

Loading file "/home/alumne/pti-master-microservices/microservi... | YAML | Tab Width: 8 | Ln 1, Col 1 | INS

Figura 12. Manifest de l'Ingress per a carrental.local.

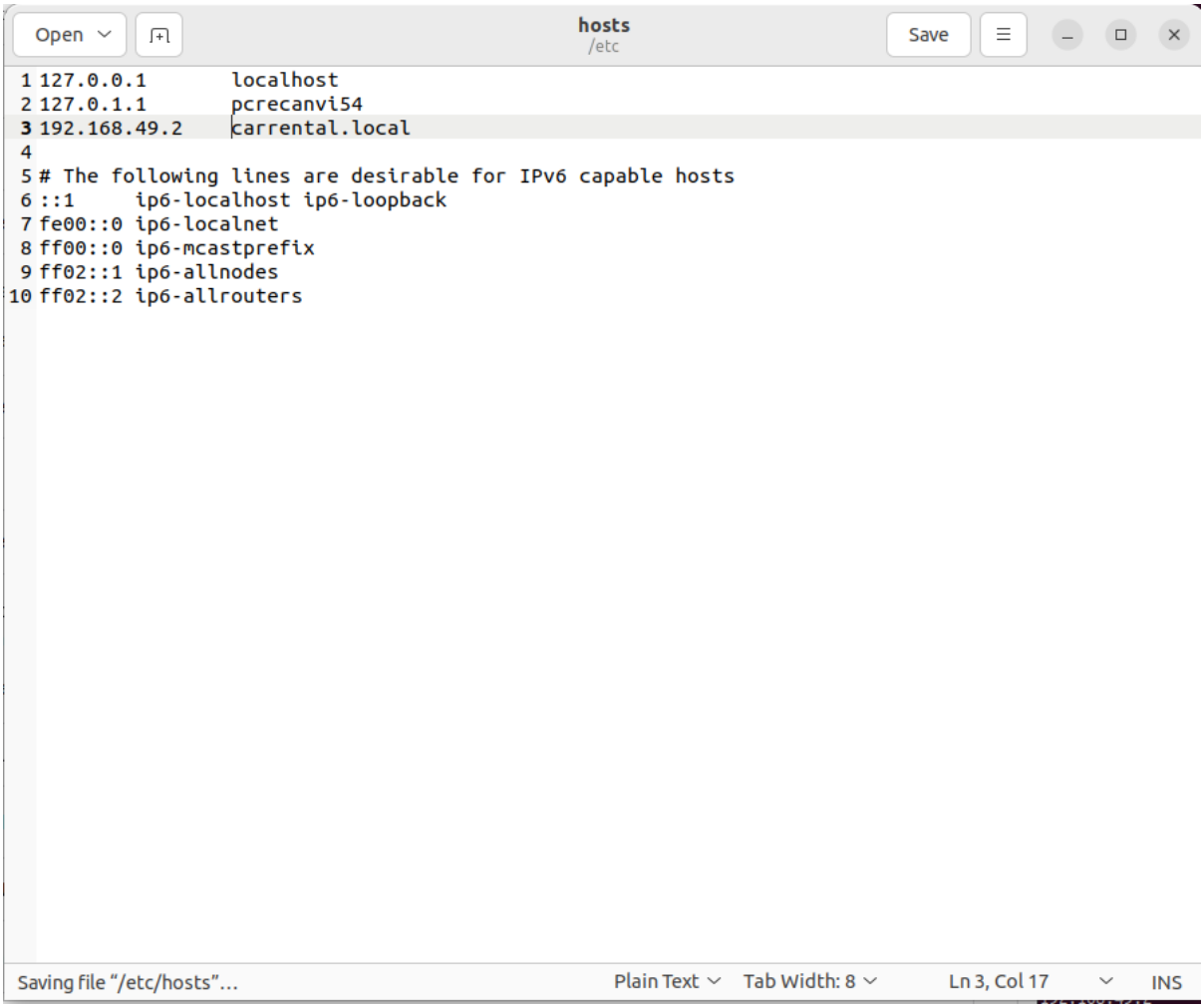


Figura 13. Entrada afegida a /etc/hosts amb la IP de Minikube.

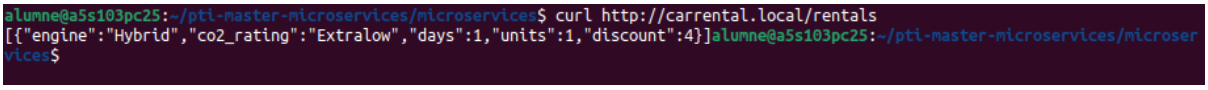


Figura 14. Petició a l'API a través de l'Ingress.

La petició a http://carrental.local/rentals retorna la llista de lloguers, de manera que l'Ingress fa de proxy invers cap al Service sense necessitat d'exposar cap NodePort.

2.6 Horizontal Pod Autoscaler (3.5)

S'ha activat metrics-server i s'han afegit requests i limits de CPU al Deployment, necessaris perquè l'HPA pugui calcular el percentatge d'ús. Després s'ha creat l'HPA amb un objectiu del 50% de CPU i entre 1 i 5 rèpliques.



Figura 15. Requests (100m) i limits (200m) de CPU afegits al Deployment.

Open *carrental-hpa.yaml ~/pti-master-microservices/microservices Save

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: carrental
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: carrental
  minReplicas: 1
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Figura 16. Manifest de l'HPA.

```
alumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl get hpa carrental
kubectl describe hpa carrental
NAME        REFERENCE              TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
carrental   Deployment/carrental   cpu: 1%/50%   1         5         3          52s
Name:                                                  carrental
Namespace:                                             default
Labels:                                                <none>
Annotations:                                           <none>
CreationTimestamp:                                     Tue, 29 Sep 2026 16:28:34 +0200
Reference:                                             Deployment/carrental
Metrics:                                               ( current / target )
  resource cpu on pods  (as a percentage of request):  1% (1m) / 50%
Min replicas:                                          1
Max replicas:                                          5
Deployment pods:                                       3 current / 3 desired
Conditions:
  Type            Status  Reason               Message
  ----            ------  ------               -------
  AbleToScale     True    ScaleDownStabilized  recent recommendations were higher than current one, applying the highest recent re
commendation
  ScalingActive   True    ValidMetricFound     the HPA was able to successfully calculate a replica count from cpu resource utiliz
ation (percentage of request)
  ScalingLimited  False   DesiredWithinRange   the desired count is within the acceptable range
Events:           <none>
```

Figura 17. Estat inicial de l'HPA: 1% d'ús de CPU i 3 rèpliques.

Per generar càrrega s'ha fet servir hey amb 40 peticions en paral·lel durant un minut:

```
hey -z 1m -c 40 http://carrental.local/rentals
```

```
^Calumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl get hpa carrental --watch
NAME        REFERENCE              TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
carrental   Deployment/carrental   cpu: 1%/50%   1         5         3          3m3s
carrental   Deployment/carrental   cpu: 98%/50%   1         5         3          4m15s
carrental   Deployment/carrental   cpu: 98%/50%   1         5         5          4m30s
```

Figura 18. Escalat cap amunt: amb el 98% de CPU les rèpliques passen de 3 a 5.

```
^Calumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl get pods --watch
NAME                          READY   STATUS    RESTARTS   AGE
carrental-56695c8498-bgr26    1/1     Running   0          9m41s
carrental-56695c8498-kxgt2    1/1     Running   0          9m41s
carrental-56695c8498-xrh46    1/1     Running   0          9m42s
helloworld-dbcdbc7cd-9f88l    1/1     Running   0          66m
helloworld-dbcdbc7cd-cgj4z    1/1     Running   0          66m
helloworld-dbcdbc7cd-trwwn    1/1     Running   0          66m
helloworld-dbcdbc7cd-vllmz    1/1     Running   0          66m
carrental-56695c8498-zmxtz    0/1     Pending   0          0s
carrental-56695c8498-zmxtz    0/1     Pending   0          0s
carrental-56695c8498-tn5ts    0/1     Pending   0          0s
carrental-56695c8498-tn5ts    0/1     Pending   0          0s
carrental-56695c8498-zmxtz    0/1     ContainerCreating   0          0s
carrental-56695c8498-tn5ts    0/1     ContainerCreating   0          0s
carrental-56695c8498-tn5ts    0/1     ContainerCreating   0          0s
carrental-56695c8498-zmxtz    0/1     ContainerCreating   0          0s
carrental-56695c8498-tn5ts    1/1     Running             0          0s
carrental-56695c8498-zmxtz    1/1     Running             0          0s
```

Figura 19. Creació dels nous Pods (Pending → ContainerCreating → Running).

```
^Calumne@a5s103pc25:~/pti-master-microservices/microservices$ kubectl get hpa carrental --watch
NAME         REFERENCE               TARGETS        MINPODS   MAXPODS   REPLICAS   AGE
carrental    Deployment/carrental    cpu: 1%/50%    1         5         3          3m3s
carrental    Deployment/carrental    cpu: 98%/50%   1         5         3          4m15s
carrental    Deployment/carrental    cpu: 98%/50%   1         5         5          4m30s
carrental    Deployment/carrental    cpu: 160%/50%  1         5         5          5m15s
carrental    Deployment/carrental    cpu: 10%/50%   1         5         5          6m15s
carrental    Deployment/carrental    cpu: 2%/50%    1         5         5          7m15s
carrental    Deployment/carrental    cpu: 1%/50%    1         5         5          8m15s
```

Figura 20. Evolució completa: pic del 160% i posterior baixada de la CPU fins a l'1%.

Durant la càrrega, l'ús de CPU arriba al 98% i fins al 160% del request, i l'HPA escala fins al màxim de 5 rèpliques. Quan acaba la càrrega, la CPU baixa a l'1% però les rèpliques es mantenen a 5, ja que Kubernetes espera 5 minuts per defecte abans de reduir-les, per evitar oscil·lacions. Passat aquest temps, les rèpliques es reduirien progressivament.

2.7 Valoració

Kubernetes facilita molt el desplegament de microserveis: les rèpliques, el balanceig de càrrega i la recuperació de Pods són automàtics, i tot es pot definir de manera declarativa amb YAML. L'Ingress permet exposar diversos serveis amb un únic punt d'entrada, i l'HPA adapta els recursos a la demanda sense intervenció manual.

Com a limitacions, guardar les dades en fitxers dins dels Pods no és adequat, ja que es perden i no es comparteixen entre rèpliques. A més, Minikube és un clúster d'un sol node, útil per aprendre però no per a producció, i l'HPA reacciona amb un cert retard tant en escalar com en reduir.